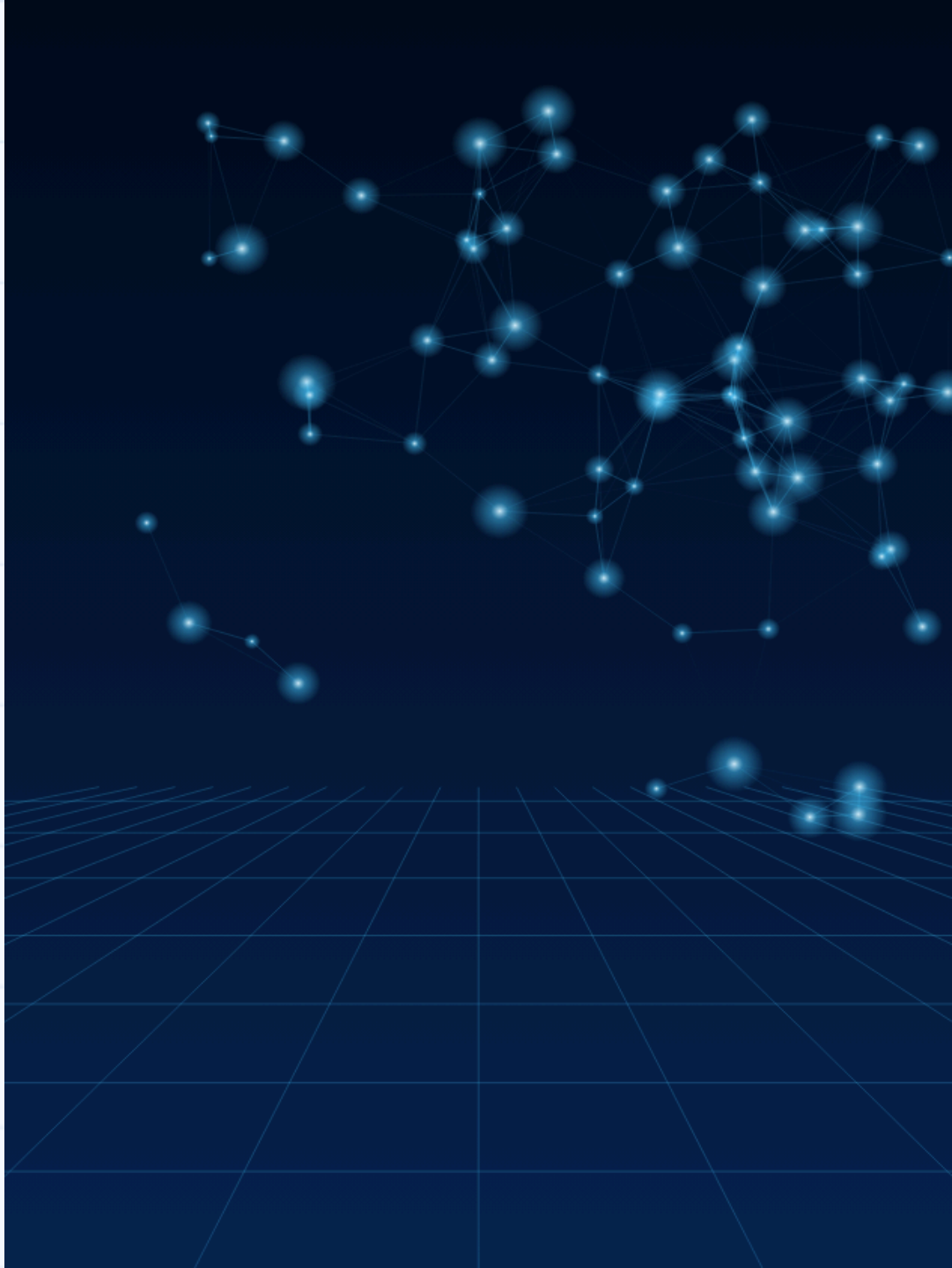
TRABALHO AV2

# Inteligência Artificial Computacional

Aprendizado de Máquina: Classificação e Regressão

Alunos: Rodrigo Herrmann e Gustavo Furtado
Professora: Dra. Cynthia Moreira Maia — UNIFOR
Python + NumPy



# 1. Introdução

## Objetivo: comparar algoritmos supervisionados implementados do zero em um problema de classificação e um de regressão do OpenML.

### O QUE COMPARAMOS

Classificação: kNN (Euclidiana e Manhattan) e Bayes Univariado.

Regressão: kNN (Euclidiana e Manhattan) e Regressão Linear Múltipla.

### POR QUE IMPORTA

Escolher um modelo exige equilibrar qualidade preditiva e custo computacional (tempo de treino e de teste). Implementar do zero, expõe a matemática de cada método.

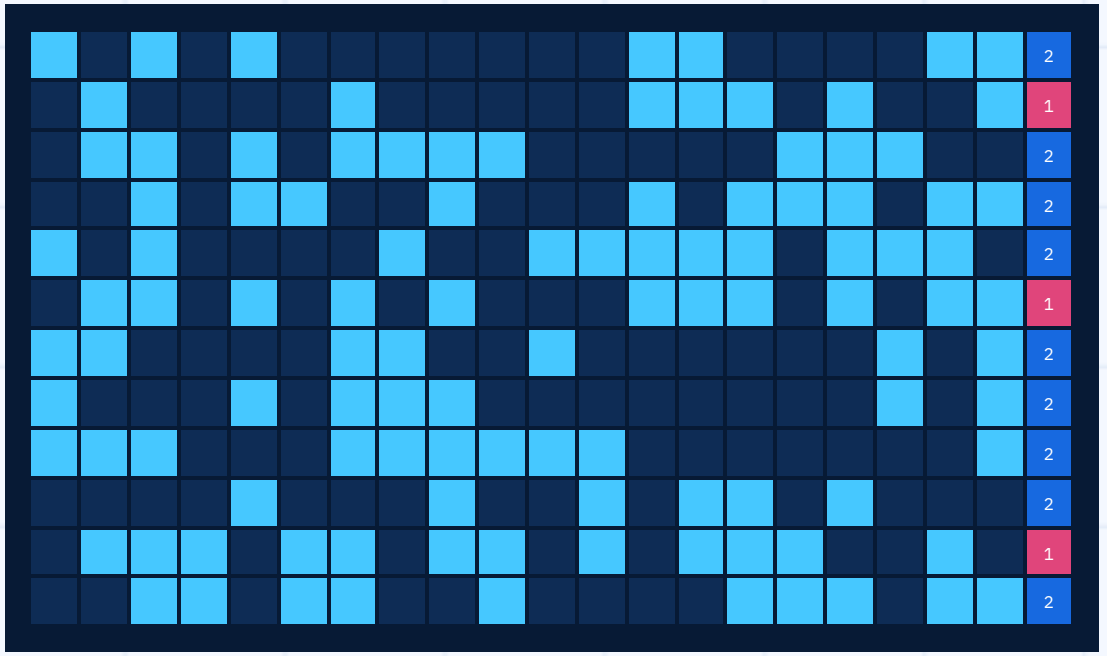
# 3.1 Banco de Dados — Classificação

## AutoUniv-au1-1000

OpenML ID 1547

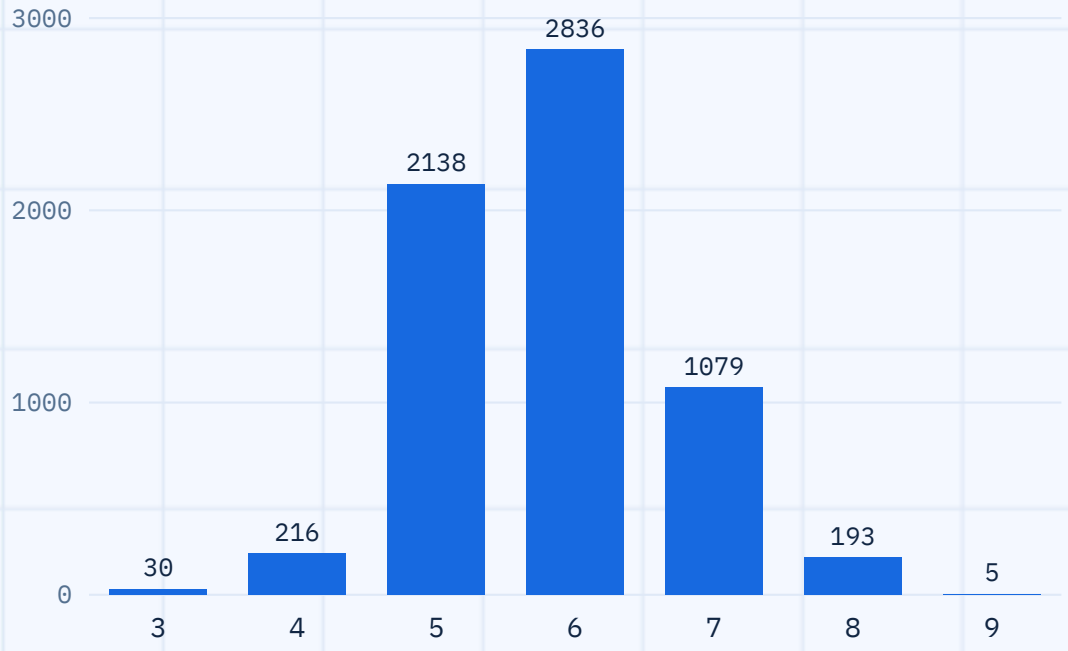
Dataset artificial do gerador AutoUniv (Ray J. Hickey): regras aleatórias com ruído controlado geram exemplos rotulados. Atributos binários (0/1), sem valores ausentes. Classes desbalanceadas: class1 = 25,9% e class2 = 74,1%.

**1.000 AMOSTRAS · 20 ATRIBUTOS · 2 CLASSES**




As 12 primeiras linhas reais do arquivo: cada quadrado é um atributo (azul-claro = 1), a última coluna é a classe.

# 3.1 Banco de Dados — Regressão




Quantidade de vinhos por nota de qualidade (6.497 amostras).

**Wine Quality**

OpenML ID 287

Prever a nota de qualidade (3 a 9) de vinhos a partir de 11 medidas físico-químicas: acidez, açúcar, cloretos, $SO_2$, densidade, pH, sulfatos, álcool etc.

6.497 AMOSTRAS · 11 ATRIBUTOS · alvo quality (média 5,82 ± 0,87)

Cortez et al., 2009

## 2. Algoritmos — kNN

# k vizinhos.
# Uma decisão.

Euclidiana: $d(X,Y) = \sqrt{\Sigma\ (x_i - y_i)^2}$
Manhattan: $d(X,Y) = \Sigma\ |x_i - y_i|$
Classificação: voto majoritário dos k = 3 vizinhos.
Regressão: média dos k = 5 vizinhos.
Modelo “preguiçoso”: o treino só armazena os dados; o custo está na previsão.

```
# classe KNN da aula
# + parâmetro distance
model = KNN(k=3,
    task='classification',
    distance='manhattan')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

# 2. Algoritmos — Classificador Bayesiano Univariado

## Uma normal por atributo, por classe.

Densidade normal univariada: $p(x) = 1/(\sqrt{(2\pi)}\cdot\sigma) \cdot \exp[-\tfrac{1}{2}((x - \mu)/\sigma)^2]$

Treino: estima a priori $P(C_k)$, a média $\mu$ e a variância $\sigma^2$ de cada atributo por classe (assume independência condicional). Uma pequena regularização é somada à variância para evitar $\sigma^2 = 0$.

Previsão: classe com maior $\log P(C_k) + \Sigma \log p(x_j \mid C_k)$. O logaritmo evita underflow numérico.

$$P(C_k \mid x) \propto P(C_k) \cdot \prod_j \mathcal{N}(x_j ; \mu_{kj}, \sigma^2_{kj})$$

# 2. Algoritmos — Regressão Linear Múltipla

## Solução fechada, com estabilidade numérica.

Modelo: $\hat{y} = \beta_0 + \beta_1 x_1 + \ldots + \beta_p x_p$, com uma coluna de 1s em X para o intercepto.

A classe MRegression da aula resolve a equação normal com a pseudo-inversa de Moore–Penrose (np.linalg.pinv), que funciona mesmo quando $X^TX$ não é invertível.

Modelo paramétrico: o treino calcula β uma vez; a previsão é só um produto matricial. Não tem hiperparâmetro para ajustar.

$$\hat{y} = X\beta$$

$$\beta = (X^TX)^+ X^Ty$$

# 3. Experimentos — Validação Cruzada K-Fold (k = 5)




Função k_fold no estilo do train_test_split da aula (np.random.seed, np.random.permutation, math.ceil). Em cada fold: padronização Z-score ajustada só no treino → treino → teste. Para cada algoritmo: média ± desvio-padrão das métricas nos 5 folds e tempos de treino e de teste.

```
seed 42 → permutation → 5 partições → 100% dos dados testados uma vez
```

# 3.2 Métricas de Avaliação

### CLASSIFICAÇÃO

$$\text{Acurácia} = (VP + VN) / N$$

$$\text{Precisão} = VP / (VP + FP)$$

$$\text{Recall} = VP / (VP + FN)$$

$$F1 = 2 \cdot P \cdot R / (P + R)$$

Precisão, recall e F1 são médias macro entre as classes.

### REGRESSÃO

$$MAE = (1/n) \Sigma |y_i - \hat{y}_i|$$

$$R^2 = 1 - SSres / SStot$$

$$R^2aj = 1 - (1 - R^2)(n - 1)/(n - p - 1)$$

Todas implementadas manualmente com NumPy.

# 3.3 Resultados de Classificação

AutoUniv-au1-1000 · média ± desvio-padrão em 5 folds · mova os controles para ver o efeito dos hiperparâmetros

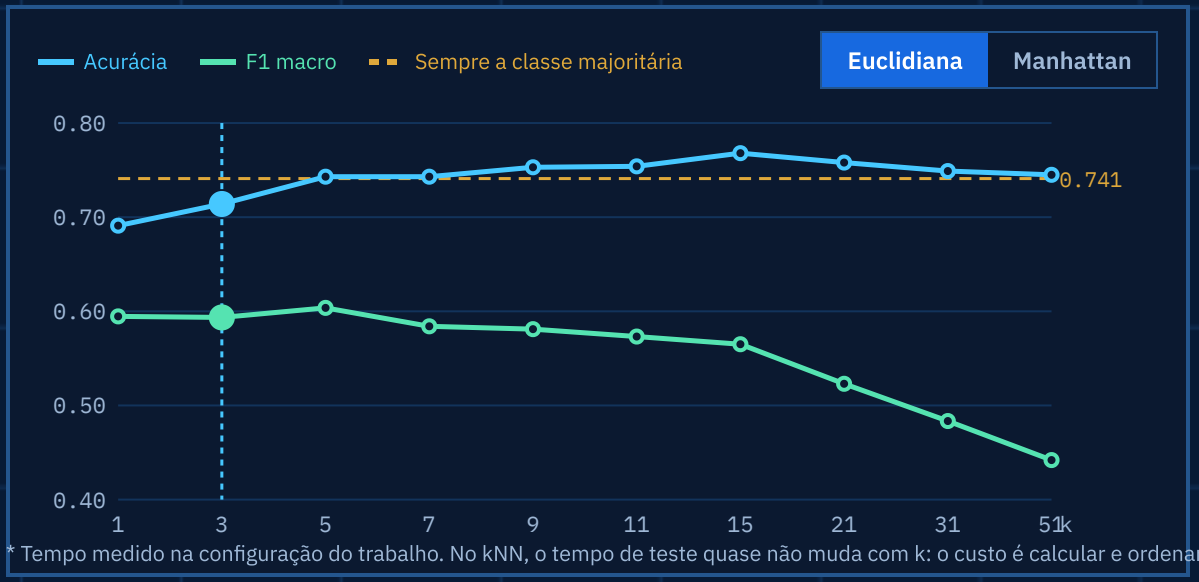
K DO KNN

k = 3

REGULARIZAÇÃO DO BAYES (SOMADA À VARIÂNCIA)

1e-9

| CLASSIFICADOR | ACURÁCIA | PRECISÃO | RECALL | F1-SCORE | TREINO (s) | TESTE (s)* |
|---|---|---|---|---|---|---|
| **kNN (Euclidiana)** k = 3 | 0.714 ± 0.027 | 0.609 ± 0.029 | 0.588 ± 0.017 | 0.593 ± 0.020 | 0.000 ± 0.000 | 0.475 ± 0.010 |
| **kNN (Manhattan)** k = 3 | 0.714 ± 0.027 | 0.609 ± 0.029 | 0.588 ± 0.017 | 0.593 ± 0.020 | 0.000 ± 0.000 | 0.422 ± 0.006 |
| **Bayes Univariado** | 0.722 ± 0.042 | 0.608 ± 0.068 | 0.565 ± 0.036 | 0.566 ± 0.044 | 0.001 ± 0.003 | 0.004 ± 0.000 |




### Se aumentar k (kNN Euclidiana, k = 3)

**Acurácia:** ir para k = 5 melhora `0.714 → 0.743`.
Melhor k para esta métrica: 15 (0.768)

**F1 macro:** ir para k = 5 melhora `0.593 → 0.604`.
Melhor k para esta métrica: 5 (0.604)

### Se aumentar a regularização do Bayes

**F1 macro:** ir para reg. = 1e-6 não muda `(0.566)`.
Melhor reg. para esta métrica: 1e-9 (0.566)

* Tempo medido na configuração do trabalho. No kNN, o tempo de teste quase não muda com k: o custo é calcular e ordenar as distâncias para todo o treino.

# 3.3 Resultados de Regressão

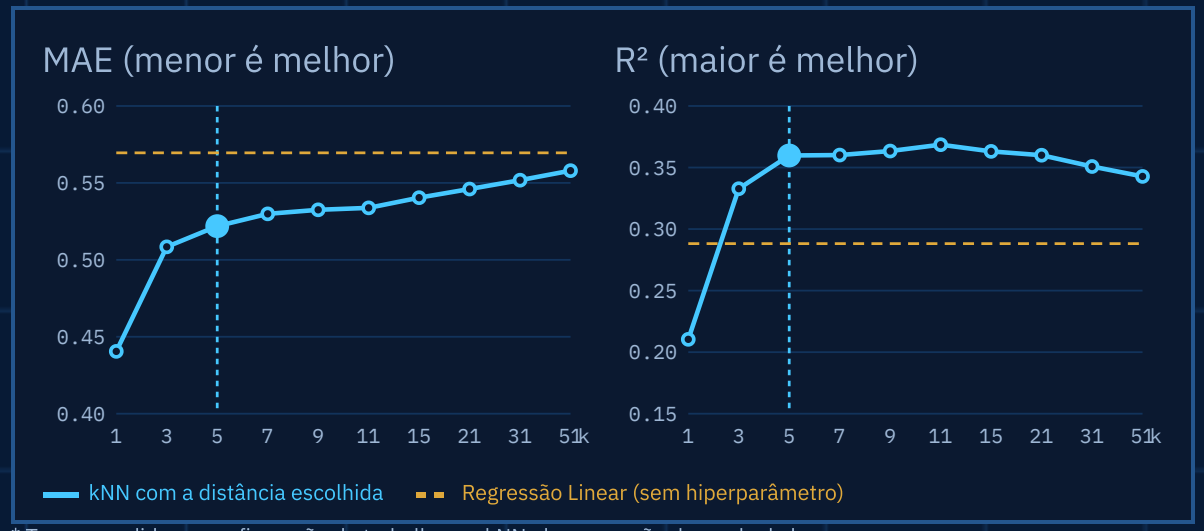
Wine Quality · média ± desvio-padrão em 5 folds · mova o controle para ver o efeito de k

K DO KNN

k = 5

Euclidiana | Manhattan

| REGRESSOR | MAE | R² | R² AJUSTADO | TREINO (s) | TESTE (s)* |
|---|---|---|---|---|---|
| **Regressão Linear** sem hiperparâmetro | 0.570 ± 0.004 | 0.288 ± 0.018 | 0.282 ± 0.018 | 0.001 ± 0.000 | 0.000 ± 0.000 |
| **kNN (Euclidiana)** k = 5 | 0.525 ± 0.006 | 0.358 ± 0.016 | 0.353 ± 0.016 | 0.000 ± 0.000 | 19.577 ± 0.120 |
| **kNN (Manhattan)** k = 5 | 0.522 ± 0.006 | 0.360 ± 0.021 | 0.354 ± 0.021 | 0.000 ± 0.000 | 17.586 ± 0.169 |




* Tempo medido na configuração do trabalho; no kNN ele quase não depende de k.

## Se aumentar k (kNN Manhattan, k = 5)

**MAE:** ir para k = 7 piora `0.522 → 0.530`.
Melhor k para esta métrica: 1 (0.441)

**R²:** ir para k = 7 não muda `(0.360)`.
Melhor k para esta métrica: 11 (0.369)

**Regressão Linear:** não tem hiperparâmetro. A pseudo-inversa dá uma única solução: MAE 0.570, R² 0.288.

# Análise Crítica: Desempenho vs. Eficiência

### CLASSIFICADORES (ACURÁCIA E F1)

Acurácia: Bayes 0.722 ≈ kNN 0.714.

F1 macro: kNN 0.593 > Bayes 0.566. O kNN acerta mais a classe minoritária.

Teste: Bayes 0.004 s vs. kNN ~0.45 s (≈100× mais rápido).

### REGRESSORES ($R^2$ E MAE)

kNN Manhattan: MAE 0.522 e $R^2$ 0.360.

Regressão Linear: MAE 0.570 e $R^2$ 0.288.

Teste: Linear < 1 ms vs. kNN ~18–20 s por fold; treino do kNN ≈ 0 s (só armazena).

**kNN: simples e sem suposições sobre os dados, mas a previsão escala mal com o tamanho do treino. Bayes e Regressão Linear: rápidos e interpretáveis, mas assumem independência entre atributos e relação linear.**

# Ajuste de Hiperparâmetros: aumentar sempre ajuda?

Não. Cada métrica tem seu melhor valor, e passar dele piora o modelo.

### k do kNN · classificação

A acurácia sobe até k = 15 (`0.768`), mas o F1 cai de `0.604` (k = 5) para `0.442` (k = 51).

Com k grande o voto é dominado pela classe majoritária (74%): o modelo acerta mais por "chutar" class2 e esquece a class1. **Melhor k pelo F1: 5.**

### k do kNN · regressão

O $R^2$ sobe de `0.21` (k = 1) até `0.369` (k = 11, Manhattan) e depois cai: média de vizinhos demais suaviza a previsão.

O MAE é menor em k = 1 (`0.441`) porque copia notas inteiras: acerta muitas exatamente e erra feio as outras, o que o $R^2$ penaliza. **Melhor k pelo $R^2$: 7 a 11.**

### Regularização do Bayes

Até `0.01` não muda nada. Com `1` ou mais, a acurácia vai para `0.741`, igual a prever sempre class2, e o F1 cai para `0.43`.

Variância inflada demais apaga as diferenças entre as classes. **Melhor: manter pequena (1e-9).**

**Aumentar k reduz a variância e aumenta o viés. O ponto ótimo depende da métrica escolhida, por isso a métrica deve ser definida antes de ajustar.**

# 4. Conclusões

## Melhor equilíbrio: Bayes Univariado e Regressão Linear.

Melhor desempenho: kNN no F1 da classificação (0.593; 0.604 com k = 5) e kNN Manhattan na regressão (MAE 0.522, $R^2$ 0.360; $R^2$ 0.369 com k = 11).

Mais rápidos: Bayes Univariado (0.004 s de teste) e Regressão Linear (< 1 ms).

Vale a pena? Na classificação, o Bayes entrega acurácia equivalente ~100× mais rápido. Na regressão, o kNN ganha +0.07 de $R^2$ ao custo de ~18 s por fold; compensa só se a previsão não precisar ser em tempo real.

Limitações: dataset de classificação artificial, ruidoso e desbalanceado (prever sempre a classe majoritária daria 0.741 de acurácia); $R^2$ moderado no Wine Quality; k escolhido sem validação aninhada.

# 5. Referências

OPENML. Dataset 1547 (AutoUniv-au1-1000) e Dataset 287 (Wine Quality). Disponível em: https://www.openml.org/

CORTEZ, P. et al. Modeling wine preferences by data mining from physicochemical properties. Decision Support Systems, 47(4), 547–553, 2009.

HARRIS, C. R. et al. Array programming with NumPy. Nature, 585, 357–362, 2020.

BISHOP, C. M. Pattern Recognition and Machine Learning. Springer, 2006.

MAIA, C. M. Material e códigos da disciplina Inteligência Artificial Computacional (kNN, regressão linear múltipla, hold-out). UNIFOR – CCT, 2026.

---

Python + NumPy